

Дорожная карта внедрения CI/CD-развертывания для mock-django

ЦБС Петроградского района - перенос схемы GitLab CI/CD / Docker Compose

Цель документа

Сформировать практическую дорожную карту переноса схемы развертывания из GitLab-проекта prog-6-ml-service на Django-проект mock-django для ЦБС Петроградского района. Под переносом понимается внедрение управляемого CI/CD-контура: проверка кода, сборка Docker-образа, публикация в реестр, развертывание через Docker Compose, контроль здоровья сервиса, регламент отката и эксплуатационные процедуры.

Ограничения и допущения

- Содержимое .gitlab-ci.yml исходного GitLab-проекта не было доступно как текст, но публичная карточка проекта подтверждает наличие CI/CD configuration и Container Registry. Поэтому ниже зафиксирована переносимая схема GitLab CI/CD, совместимая с видимым составом mock-django.
- В mock-django уже есть Dockerfile, docker-compose.yml, Gunicorn, PostgreSQL и endpoint /health/; это позволяет внедрять развертывание без смены технологического стека.
- Целевой вариант: зеркалировать или перенести mock-django в GitLab, подключить GitLab Runner, GitLab Container Registry и деплой на сервер ЦБС по SSH. Альтернатива: сохранить GitHub как основной репозиторий и использовать GitLab mirror только для CI/CD.

Целевая схема развертывания

Уровень	Что внедряется	Назначение
Repository	GitLab-проект или mirror GitHub -> GitLab	Единая точка запуска pipeline, merge request, protected branches, аудит изменений.
CI	stages: lint/test/build/deploy	Автоматические проверки Django, сборка образа, публикация в registry, управляемый деплой.
Registry	GitLab Container Registry	Хранение версионированных Docker-образов: SHA, branch, release tags.
Runtime	Docker Compose на сервере ЦБС	web + db + reverse proxy/nginx-proxy; поддержка healthcheck и rollback.
Secrets	GitLab CI/CD variables + .env на сервере	Исключение SECRET_KEY, DB-паролей, SSH-ключей и доменных имен из репозитория.
Control	/health/, docker compose ps, logs, backups	Проверка доступности, диагностика, восстановление.

Роли и зоны ответственности

Роль	Ответственность	Минимальные решения/доступы
Владелец продукта ЦБС	Приоритизация внедрения, приемка, согласование окна релиза.	Право принять MVP-развертывание и rollback-план.
Разработчик Django	Исправления settings.py, тесты, миграции, healthcheck, README_DEPLOY.	Commit/MR в репозитории, code review.
DevOps/инженер CI/CD	GitLab Runner, registry, .gitlab-ci.yml, docker-compose.prod.yml, deploy.sh.	Maintainer в GitLab, доступ к CI/CD variables.
Администратор сервера	Docker/Compose, firewall, домен/TLS, резервные копии, системные логи.	SSH-доступ, sudo для deploy-пользователя, сетевые правила.
Ответственный за ИБ/персональные данные	Проверка секретов, прав доступа, протоколов, журналирования.	Право утвердить production-секреты и доступы.

Дорожная карта работ

№	Этап	Организационные действия	Технические действия	Результат/критерий готовности
1	Инициация и доступы	Назначить владельца, DevOps, разработчика и администратора. Утвердить окружения: dev/staging/prod. Выбрать вариант: перенос в GitLab или mirror из GitHub.	Создать GitLab-проект/группу, protected branches main и release/*, подготовить deploy-user на сервере.	Есть владелец, hero/mirror, матрица доступов, согласованное окно пилотного релиза.
2	Инвентаризация mock-django	Зафиксировать текущий запуск и риски: секреты в коде, wildcard ALLOWED_HOSTS, отсутствие тестов/миграционного регламента.	Проверить Dockerfile, compose, requirements, settings.py, endpoint /health/, схему БД и статические файлы.	Сформирован backlog hardening-задач и список переменных окружения.
3	Подготовка application hardening	Согласовать доменное имя, HTTPS, правила журналирования и резервного копирования БД.	SECRET_KEY из env, ALLOWED_HOSTS из env, CSRF/SESSION secure cookies, STATIC_ROOT, collectstatic, manage.py check --deploy, базовые pytest/Django tests.	Проект готов к production-конфигурации без секретов в Git.
4	Контейнеризация и compose-prod	Определить политику хранения БД: managed DB или локальный postgres volume. Утвердить retention backups.	Создать docker-compose.prod.yml с image из registry, db healthcheck, web healthcheck/curl, env_file, restart policy, отдельными volumes.	Сервис запускается на staging командой docker compose -f docker-compose.prod.yml up -d.
5	CI pipeline	Утвердить правила: деплой staging автоматом из main, production - вручную по tag/release.	Добавить .gitlab-ci.yml: lint, test, build-image, push registry, deploy-staging, deploy-production manual. Настроить GitLab variables.	Pipeline проходит до staging без ручного доступа к серверу.
6	Сервер и network	Согласовать IP/домен, firewall, TLS-сертификаты, доступы администратора, учетную запись deploy.	Установить Docker Engine/Compose, настроить SSH key, директорию /opt/mock-django, .env, docker login, reverse proxy.	Сервер принимает деплой; health URL отвечает 200; БД готова.
7	Пилотный релиз	Провести релизное окно, уведомить пользователей/заказчика, подготовить коммуникацию о rollback.	Запустить pipeline, применить migrations, collectstatic, выполнить smoke tests: /, /health/, docker logs, rollback rehearsal.	Staging/production доступны, smoke tests пройдены, rollback проверен.
8	Эксплуатация и развитие	Закрепить SLA, владельца мониторинга, график регулярных обновлений зависимостей.	Добавить backup job, логирование, алерты, dependency scanning, release notes, auto cleanup registry tags.	Есть регламент сопровождения, приемочный чек-лист и документация.

Backlog технических задач

Приоритет	Задача	Описание	Критерий приемки
P0	Секреты и настройки через env	SECRET_KEY, ALLOWED_HOSTS, POSTGRES_*, DEBUG, домен, secure cookies вынести в переменные окружения/GitLab CI/CD variables.	В репозитории нет production-секретов; приложение стартует с .env.
P0	CI/CD pipeline	Автоматическая проверка, сборка образа, push в registry, deploy на staging; production manual.	MR не мерджится при падающих checks; deploy воспроизводим.
P0	Healthcheck и readiness	Оставить /health/ как внешний endpoint; добавить healthcheck в compose для db и web.	pipeline/compose ждут готовность БД и web; smoke test фиксирует 200/500.
P1	Тесты	Минимум: test для / и /health/ с mock/тестовой БД, плюс django check --deploy.	CI job test стабильно проходит.
P1	Статика и миграции	STATIC_ROOT, collectstatic, migrate как job/entrypoint с контролем ошибок.	После деплоя статические файлы собраны, миграции применены.
P1	Наблюдаемость	docker logs, ротация логов, status page/alerts, регулярные backup БД.	Есть инструкции диагностики и восстановления.
P2	Усиление Dockerfile	non-root user, pinned dependencies, build cache, зависимость curl для healthcheck, минимизация образа.	Образ проходит security review, запускается без root при возможности.

Контрольные точки и результаты

- M1: репозиторий в GitLab/mirror, доступы и protected branches готовы.
- M2: настройки Django и секреты приведены к production-формату.
- M3: CI собирает и публикует Docker image в registry.
- M4: staging-деплой выполняется автоматически, healthcheck проходит.
- M5: production-деплой ручной, с утвержденным rollback-планом и backup.

- М6: эксплуатационный регламент передан администратору и владельцу продукта.

Источники и проверенные факты

Дата подготовки: 20 июня 2026 г. Источники использованы для инвентаризации репозиторий и обоснования технических решений.

- GitLab: <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service> - публичная карточка проекта: Project ID 399, 29 commits, 2 branches, наличие CI/CD configuration, Container Registry, файл .gitlab-ci.yml 2.86 KB; содержимое .gitlab-ci.yml не было доступно как текст без авторизации/рендера.
- GitHub: <https://github.com/schudnaya/mock-django> - публичный репозиторий mock-django, 10 commits, файлы app, Dockerfile, README.md, docker-compose.yml, requirements.txt; README содержит локальный запуск через docker compose up -d --build и переход на <http://localhost:8080>.
- Dockerfile mock-django: Python 3.11 Alpine, установка системных пакетов, pip install -r requirements.txt, запуск Gunicorn config.wsgi:application на 0.0.0.0:8000.
- docker-compose.yml mock-django: сервисы web, db(PostgreSQL 15 Alpine), nginx-proxy; порт хоста 8080 -> 80; переменные POSTGRES_* заданы в compose.
- Django settings.py mock-django: SECRET_KEY задан в коде, DEBUG=False, ALLOWED_HOSTS=["*"], подключение к PostgreSQL через переменные окружения; URL /health/ выполняет SELECT 1 к БД.
- GitLab Docs: .gitlab-ci.yml описывает jobs pipeline; predefined variables доступны в каждом pipeline; CI/CD variables применяются для переиспользования значений и предотвращения hard-code секретов; GitLab CI/CD может строить, тестировать и push Docker images.
- Docker Docs: Compose создает сервисы в порядке зависимостей; healthcheck/depends_on с condition: service_healthy позволяют ждать готовности зависимого сервиса.
- Django Docs: deployment checklist рекомендует manage.py check --deploy, секретный SECRET_KEY вне source control, DEBUG=False в production, конкретные ALLOWED_HOSTS, STATIC_ROOT/collectstatic, HTTPS и secure cookies.